Day 86 - GitOps Project: End-to-End CI/CD Pipeline with AI-BankApp

1. What is a CI/CD Pipeline?

A **CI/CD pipeline** is an automated workflow that takes your code from development to production.

It mainly has two parts:

Continuous Integration (CI)

Whenever a developer pushes code to Git,

- Source code is downloaded
- Dependencies are installed
- Application is compiled
- Tests are executed
- Docker image is built
- Docker image is pushed to a container registry

The purpose is to ensure every code change is tested and packaged automatically.

Continuous Deployment (CD)

After CI completes,

- Kubernetes manifests are updated
- Git repository is updated
- ArgoCD detects the Git change
- ArgoCD deploys the new version into Kubernetes

No one manually runs:

kubectl apply

Everything happens automatically.

---

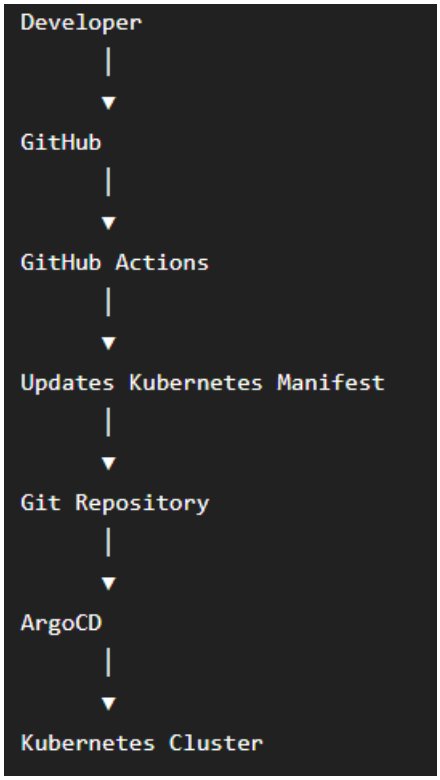
2. Difference Between Traditional CI/CD and GitOps CI/CD

Traditional CI/CD

C

I pipeline directly talks to Kubernetes.

GitOps CI/CD



Notice:

GitHub Actions **never deploys** to Kubernetes.

It only updates Git.

ArgoCD performs the deployment.

---

3. Git is the Single Source of Truth

In GitOps,

Git always contains the desired state.

Example

Git says

Replicas = 4

Image = bankapp:v12

Even if someone manually changes

Replicas = 1

ArgoCD notices the difference and restores

Replicas = 4

Everything inside Kubernetes should always match Git.

---

4. GitHub Actions

GitHub Actions is GitHub's automation platform.

It runs workflows stored in

.github/workflows/

Whenever an event occurs such as

- Push
- Pull Request
- Tag creation
- Manual trigger

GitHub automatically starts a virtual machine and executes every step in the workflow.

Example

Push Code

↓

Start Ubuntu VM

↓

Checkout Repository

↓

Install Java

↓

Compile Application

↓

Run Tests

↓

Build Docker Image

↓

Push Docker Image

↓

Update Deployment YAML

↓

Commit Changes

↓

Push Back to Git

---

5. Workflow Trigger

Every GitHub Actions workflow begins with a trigger.

Example

```
on:
  push:
```

branches:

- feat/gitops

Meaning

Whenever code is pushed to

feat/gitops

the workflow starts automatically.

The AI-BankApp also uses

paths:

Example

paths:

- src/
- Dockerfile
- pom.xml

Meaning

Only run the workflow if these files change.

If only Kubernetes manifests change,

the workflow will not execute.

This prevents unnecessary builds.

---

6. GitHub Runner

A **Runner** is the temporary machine that executes the workflow.

Example

runs-on: ubuntu-latest

GitHub creates

Ubuntu VM

↓

Runs every workflow step

↓

Deletes VM after completion

Every workflow starts with a fresh environment.

---

7. GitHub Secrets

Never store passwords inside code.

Instead, GitHub provides encrypted Secrets.

Example

DOCKERHUB_USERNAME

DOCKERHUB_TOKEN

AWS_ACCESS_KEY_ID

AWS_SECRET_ACCESS_KEY

Used like

${{ secrets.DOCKERHUB_TOKEN }}

The actual password is never exposed in logs.

---

8. DockerHub Access Token

Instead of logging in using your Docker password,

DockerHub recommends using an Access Token.

Benefits

- More secure
- Can be revoked anytime
- Supports automation
- Doesn't expose your password

GitHub Actions uses this token to push Docker images.

---

9. Docker Image Tagging

Every Docker image should have a version.

Example

latest

v1.0

v2.1

a4f2bc1

AI-BankApp uses Git commit SHA.

Example

Karinasayta1/ai-bankapp-eks

↓

latest

↓

4bd3ac2

Every commit produces a unique Docker image.

Benefits

- Easy rollback
- Easy debugging
- Every deployment is traceable

---

10. Git Commit SHA

Every Git commit has a unique identifier.

Example

Full SHA

5e8d7af23412cce.....

↓

Short SHA

5e8d7af

The workflow runs

git rev-parse --short HEAD

to generate the image tag.

Example

bankapp:5e8d7af

---

11. Why Update Kubernetes Manifest Automatically?

Suppose GitHub built

bankapp:v25

Kubernetes is still using

bankapp:v24

The deployment manifest must be updated.

Example

Before

image: bankapp:v24

After

image: bankapp:v25

This change is committed back into Git.

ArgoCD notices this commit and deploys the new version.

---

12. Why [skip ci] is Used

Imagine this sequence

Pipeline updates

deployment.yaml

↓

Pipeline commits changes

↓

Git receives new commit

↓

Pipeline starts again

↓

Updates deployment

↓

Commits again

↓

Pipeline starts again

This becomes an infinite loop.

To prevent this,

GitHub Actions commits using

[skip ci]

Example

ci: update image to a4f2bc [skip ci]

GitHub ignores commits containing [skip ci].

---

13. ArgoCD Continuous Synchronization

ArgoCD continuously checks

Git

↓

Cluster

If both are identical

Synced

If different

OutOfSync

When

selfHeal: true

ArgoCD automatically restores the cluster to match Git.

---

14. Rolling Update

When a new image is deployed,

Kubernetes does not stop all pods at once.

Instead

Old Pod

↓

New Pod Created

↓

Health Check

↓

Traffic Shifted

↓

Old Pod Deleted

Benefits

- Zero downtime
- Users continue accessing the application
- Safer deployments

---

15. Drift Detection

Configuration Drift means

Cluster configuration no longer matches Git.

Example

Git

Replicas = 4

Someone manually executes

kubectl scale deployment bankapp --replicas=1

Now

Git

4

Cluster

1

ArgoCD detects this difference and restores the deployment.

---

16. Tools Used in Day 86

Tool	Purpose
Git	Stores application source code and Kubernetes manifests
GitHub	Hosts the Git repository and triggers workflows
GitHub Actions	Builds, tests, packages the application, pushes Docker image, and updates manifests
Maven	Builds the Java Spring Boot application
Docker	Packages the application into a container image
DockerHub	Stores versioned Docker images
Kubernetes	Runs the application containers
Amazon EKS	Managed Kubernetes cluster on AWS
ArgoCD	Watches Git and continuously deploys the desired state to Kubernetes

These concepts provide the foundation needed to understand every task in **Day 86**, from the GitHub Actions workflow through automated image builds, manifest updates, ArgoCD synchronization, drift detection, and the complete end-to-end GitOps deployment pipeline.

Task 1: Study the AI-BankApp's GitOps CI Pipeline

Open `.github/workflows/gitops-ci.yml` from the AI-BankApp repo. This is a production-grade GitOps CI pipeline.

The workflow triggers on:

yaml

```
on:
  push:
    branches: [feat/gitops]
    paths:
      - 'src/'
      - 'pom.xml'
      - 'Dockerfile'
  workflow_dispatch:
```

It only runs when application code changes (`src/`, `pom.xml`, `Dockerfile`) -- not when Kubernetes manifests change. This prevents infinite loops since the pipeline itself updates manifests.

The pipeline steps:

Step	What it does
Checkout code	Clones the repo
Set up JDK 21	Installs Java 21 with Maven cache
Build with Maven	`./mvnw clean package -DskipTests -B`
Run tests	`./mvnw test -B` (non-blocking: `continue-on-error: true`)
Set image tag	Uses `git rev-parse --short HEAD` as the tag (e.g., `1c7cb0e`)
Login to DockerHub	Authenticates with secrets
Build and push image	Pushes `trainwithshubham/ai-bankapp-eks:latest` and `:sha`
Update K8s manifest	Uses `sed` to update the image tag in `k8s/bankapp-deployment.yml`
Commit updated manifest	Commits the change with `[skip ci]` to avoid re-triggering

The critical GitOps step is the last two:

yaml

```
- name: Update Kubernetes deployment manifest
```

```
  run: |
    sed -i "s|image: ${{ env.DOCKERHUB_REPO }}:.|image: ${{ env.DOCKERHUB_REPO }}:${{ steps.tag.outputs.sha_short }}|" k8s/bankapp-deployment.yml

- name: Commit updated manifest
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "github-actions[bot]@users.noreply.github.com"
    git add k8s/bankapp-deployment.yml
    git diff --staged --quiet || git commit -m "ci: update bankapp image to ${{ steps.tag.outputs.sha_short }} [skip ci]"
    git push
```

Why `[skip ci]`? Without it, the commit that updates the manifest would trigger the pipeline again, which would update the manifest again -- an infinite loop. `[skip ci]` tells GitHub Actions to ignore this commit.

The handoff to ArgoCD:

```
GitHub Actions commits new image tag to k8s/bankapp-deployment.yml
          |
    ArgoCD detects the new commit (within 3 minutes)
          |
    ArgoCD compares: cluster has old image, Git has new image
          |
    ArgoCD syncs: performs a rolling update
          |
    New pods start with the new image, old pods terminate
          |
    Zero downtime deployment complete
```

Task 2: Set Up the Pipeline on Your Fork

To run the full pipeline, you need your own fork with GitHub Secrets.

1. Fork the repo (if not done on Day 84):

- **https://github.com/TrainWithShubham/AI-BankApp-DevOps -> Fork**

2. Create a DockerHub access token:

- Go to https://hub.docker.com/settings/security
- Create a new access token with Read/Write permissions
- Note the token

3. Add GitHub Secrets to your fork:

- Go to your fork > Settings > Secrets and variables > Actions
- Add these secrets:
- `DOCKERHUB_USERNAME` -- your DockerHub username
- `DOCKERHUB_TOKEN` -- the access token from step 2

4. Update the workflow to push to your DockerHub repo:

- Edit `.github/workflows/gitops-ci.yml` in your fork:
- yaml

  env:

  DOCKERHUB_REPO: <your-dockerhub-username>/ai-bankapp-eks

5. Update the ArgoCD Application to watch your fork:

- **argocd app set bankapp --repo https://github.com/<your-username>/AI-BankApp-DevOps.git**

6. Update the Kubernetes deployment to pull from your DockerHub:

- Edit `k8s/bankapp-deployment.yml`:
- yaml

  image: <your-dockerhub-username>/ai-bankapp-eks:latest

- Commit and push all changes to your fork's `feat/gitops` branch.

Task 3: Trigger the Full Pipeline

- Make a visible code change in the application. Edit a file in `src/`:
- For example, edit `src/main/resources/templates/fragments/layout.html` -- change the page title or footer text to include your name:

  html

  <!-- Find the title or footer and add your touch -->

  <title>AI BankApp - Built by YourName</title>

- Commit and push:

  git add src/

  git commit -m "feat: customize app title"

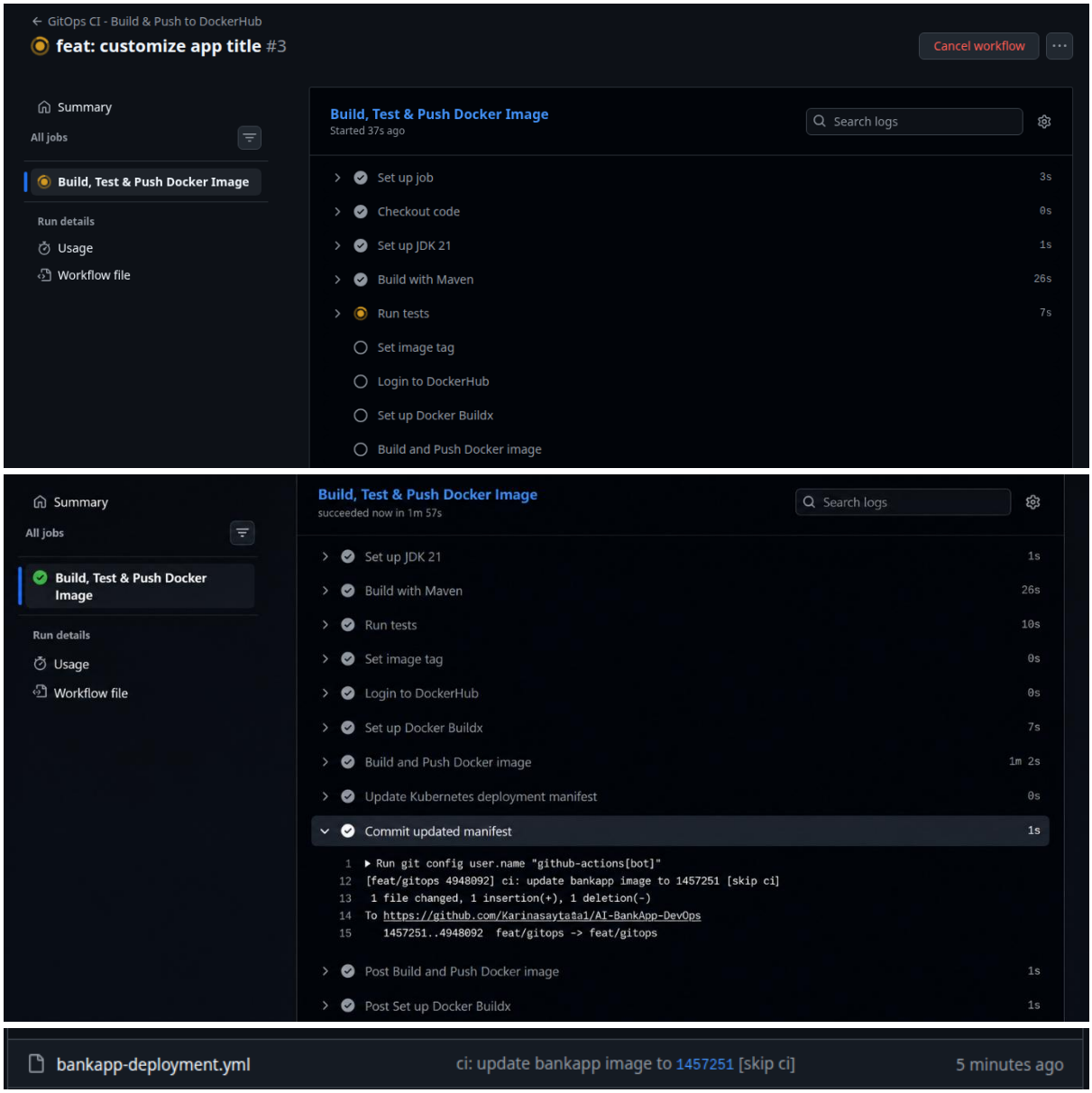
  git push origin feat/gitops

Watch the pipeline:

- Go to your fork > Actions tab
- The "GitOps CI - Build & Push to DockerHub" workflow should be running
- Watch each step: build -> test -> push -> update manifest -> commit

After the pipeline completes:

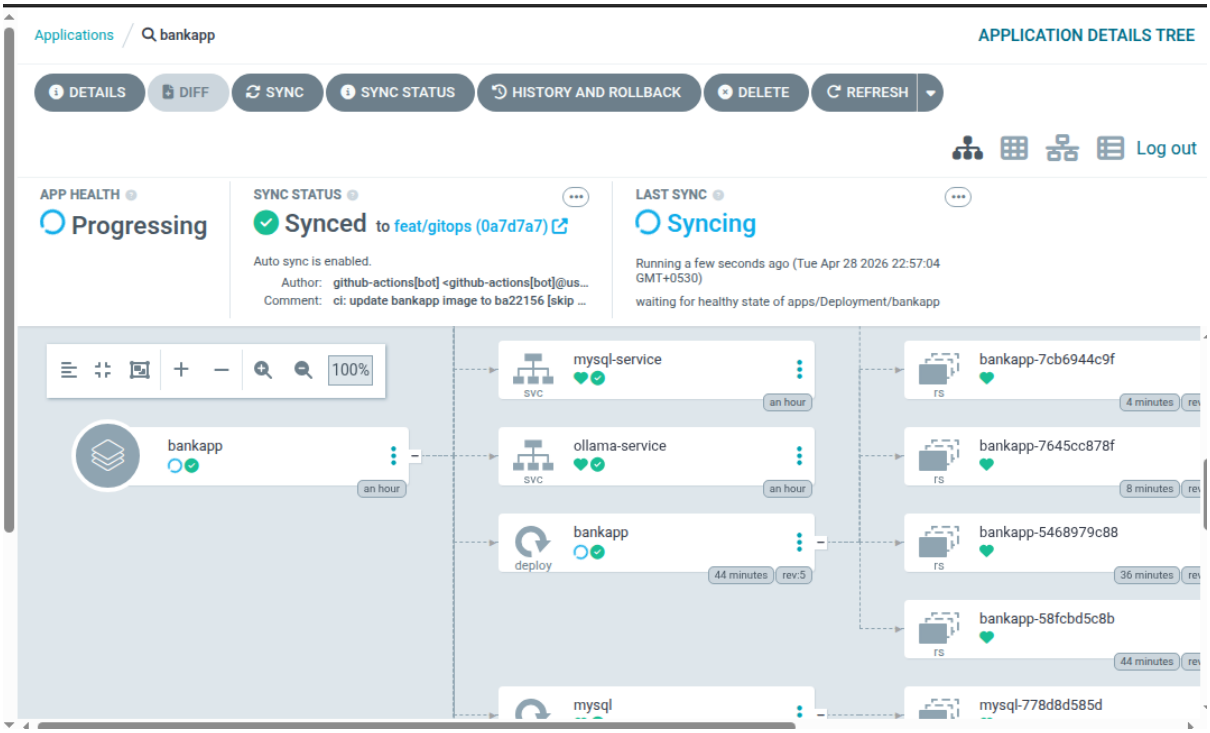
- Check the last commit on your `feat/gitops` branch -- you should see a commit from `github-actions[bot]` with the message `ci: update bankapp image to <sha> [skip ci]`
- The `k8s/bankapp-deployment.yml` file now has the new image tag



Watch ArgoCD sync:

- **argocd app get bankapp --refresh**
- **argocd app wait bankapp**

Or watch in the ArgoCD UI -- you will see a new sync event with the updated revision.



Check the pods:

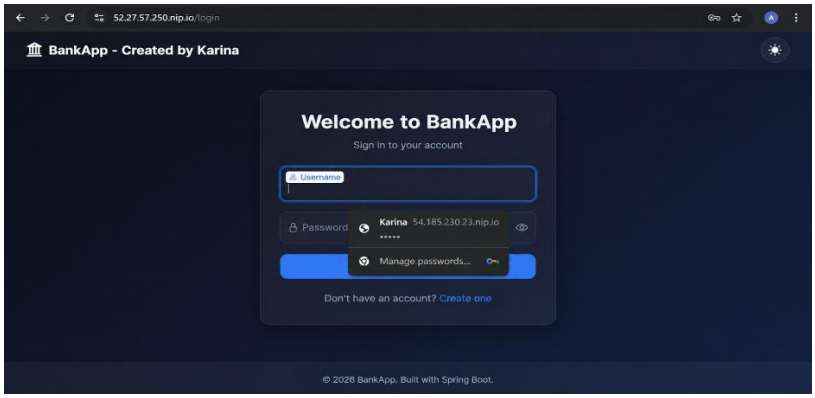
- **kubectl get pods -n bankapp -w**

```
NAME                      READY   STATUS    RESTARTS   AGE
bankapp-7cb6944c9f-9jpxl  1/1     Running   0          46s
bankapp-7cb6944c9f-9v6qg  1/1     Running   0          92s
bankapp-7cb6944c9f-hqmx8  1/1     Running   0          46s
bankapp-7cb6944c9f-tgtnx  1/1     Running   0          92s
mysql-778d8d585d-28h98    1/1     Running   0          44m
ollama-68786b495d-wx4kk   1/1     Running   0          44m
```

You should see a rolling update -- new pods starting with the new image while old pods terminate gracefully.

Verify the change is live:

- **kubectl port-forward svc/bankapp-service -n bankapp 8080:8080**



Open `http://localhost:8080` and confirm your title change is visible.

You just completed a full GitOps cycle: code change -> CI builds image -> updates manifest -> ArgoCD deploys to production. Zero manual intervention.

Task 4: Test Drift Detection and Recovery

GitOps means the cluster must always match Git. Test what happens when someone makes unauthorized changes.

Scenario 1 -- Someone scales down the app directly:

- **kubectl scale deployment bankapp -n bankapp --replicas=1**

```
karina@control-plane:~/devops/AI-BankApp-DevOps$ kubectl scale deployment bankapp -n bankapp --replicas=1
deployment.apps/bankapp scaled
```

Check ArgoCD:

- **argocd app get bankapp**

```
karina@control-plane:~/devops/AI-BankApp-DevOps$ argocd app get bankapp
Name:               argocd/bankapp
Project:            default
Server:             https://kubernetes.default.svc
Namespace:          bankapp
URL:                https://argocd.example.com/applications/bankapp
Source:
- Repo:             https://github.com/karinasayta1/AI-BankApp-DevOps.git
  Target:           feat/gitops
  Path:             k8s
SyncWindow:         Sync Allowed
Sync Policy:        Automated (Prune)
Sync Status:        OutOfSync from feat/gitops (ca18354)
Health Status:      Healthy
```

Status should show `OutOfSync`. With `selfHeal: true`, ArgoCD will correct it within 3 minutes. Monitor:

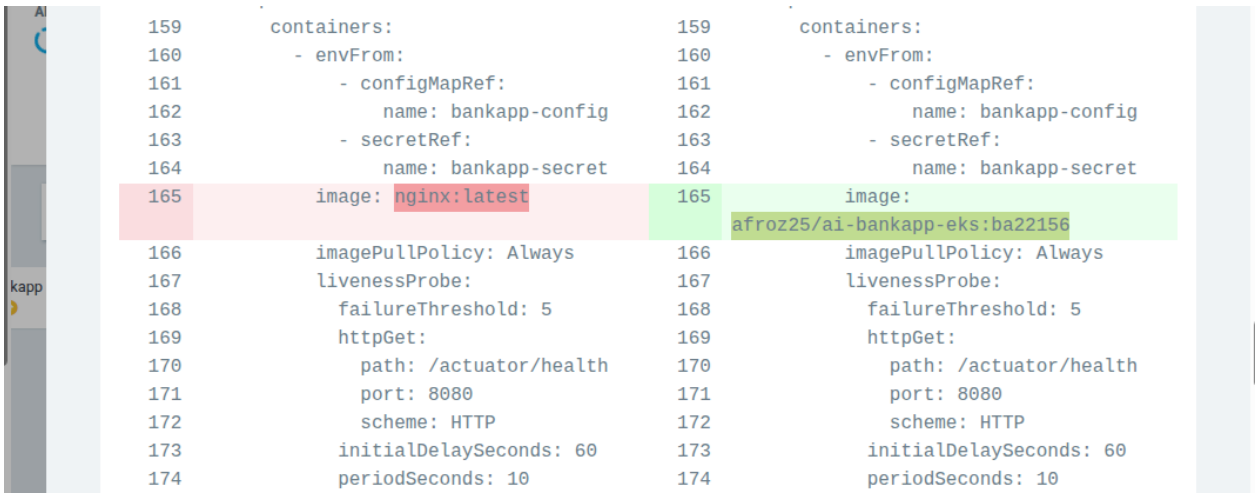
- **kubectl get pods -n bankapp -w**

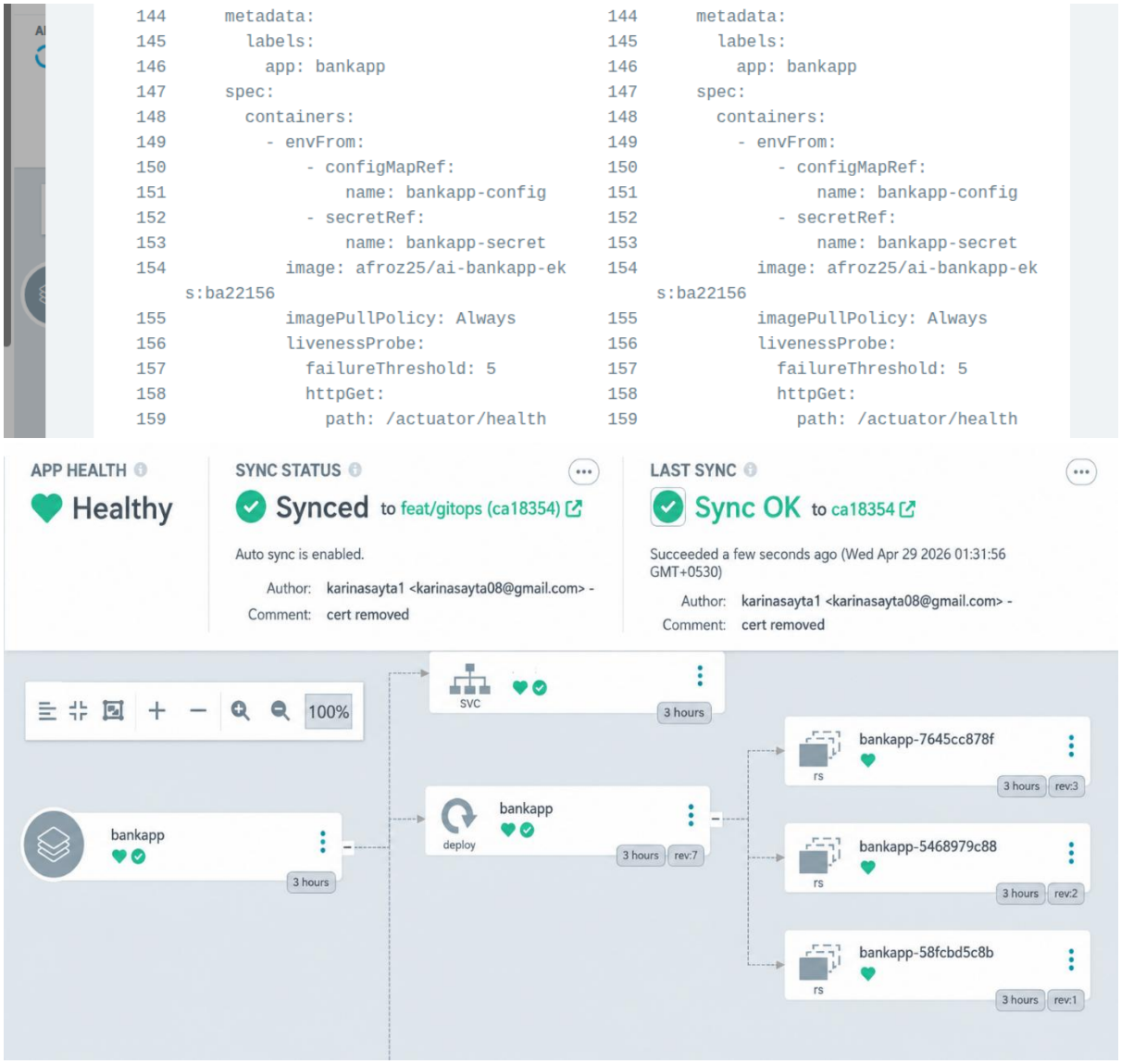
```
NAME                      READY   STATUS    RESTARTS   AGE
bankapp-b588fbf6-dlbxh    0/1     Running   0          11s
bankapp-b588fbf6-nqnb9    1/1     Running   0          147m
mysql-778d8d585d-28h98    1/1     Running   0          3h14m
ollama-68786b495d-wx4kk   1/1     Running   0          3h14m
bankapp-b588fbf6-dlbxh    1/1     Running   0          34s
bankapp-b588fbf6-6tspm    0/1     Pending   0          0s
bankapp-b588fbf6-6tspm    0/1     Pending   0          0s
bankapp-b588fbf6-6tspm    0/1     Init:0/2  0          0s
bankapp-b588fbf6-bp5ml    0/1     Pending   0          0s
bankapp-b588fbf6-bp5ml    0/1     Pending   0          0s
bankapp-b588fbf6-bp5ml    0/1     Init:0/2  0          0s
bankapp-b588fbf6-bp5ml    0/1     Init:1/2  0          1s
bankapp-b588fbf6-6tspm    0/1     Init:1/2  0          1s
bankapp-b588fbf6-bp5ml    0/1     PodInitializing   0          2s
bankapp-b588fbf6-6tspm    0/1     PodInitializing   0          2s
bankapp-b588fbf6-bp5ml    0/1     Running           0          3s
bankapp-b588fbf6-6tspm    0/1     Running           0          3s
```

The replica count will return to 4 (or whatever the manifest specifies).

Scenario 2 -- Someone updates the image tag directly:

- **kubectl set image deployment/bankapp bankapp=nginx:latest -n bankapp**





ArgoCD detects the drift and reverts it to the image tag from Git. The BankApp pods restart with the correct image.

Scenario 3 - Someone deletes a critical resource:

- **kubectl delete service bankapp-service -n bankapp**

```
karina@control-plane:~/devops/AI-BankApp-DevOps$ kubectl delete service service -n bankapp
service "bankapp-service" deleted from bankapp namespace
karina@control-plane:~/devops/AI-BankApp-DevOps$ kubectl get svc bankapp-service -n bankapp
NAME              TYPE        CLUSTER-IP       EXTERNAL-IP    PORT(S)    AGE
bankapp-service   ClusterIP   172.20.93.249    <none>         8080/TCP   20s
```

ArgoCD recreates it from Git.

View all drift events:

- **argocd app history bankapp**

```
ID      DATE                           REVISION
0       2026-04-28 22:14:03 +0530 IST  feat/gitops (acc08be)
1       2026-04-28 22:19:57 +0530 IST  feat/gitops (1457251)
2       2026-04-28 22:23:01 +0530 IST  feat/gitops (4948092)
3       2026-04-28 22:40:53 +0530 IST  feat/gitops (4948092)
4       2026-04-28 22:47:36 +0530 IST  feat/gitops (b4b74eb)
5       2026-04-28 22:50:24 +0530 IST  feat/gitops (b7f9c55)
6       2026-04-28 22:54:49 +0530 IST  feat/gitops (ba22156)
7       2026-04-28 22:58:30 +0530 IST  feat/gitops (0a7d7a7)
8       2026-04-29 01:19:57 +0530 IST  feat/gitops (ca18354)
9       2026-04-29 01:31:55 +0530 IST  feat/gitops (ca18354)
```

In the ArgoCD UI, click the application and look at the "Events" tab. Every self-heal action is logged with the before/after state.

Document: **In each scenario, how long did ArgoCD take to detect and fix the drift? What would happen if `selfHeal` was disabled?**

- It took 1-3 minitues to detect and fix the drift.
- If `selfHeal` was disabled ArgoCD would detect the drift but would not fix it automatically. We manually have to sync.

Task 5: Reflect on the Complete DevOps Pipeline

Step back and look at everything you have built across the entire 90-day challenge that connects to this GitOps pipeline:

```
[Developer writes code]
    |
[Git push to GitHub]  ..... Day 22-28: Git & GitHub
    |
[GitHub Actions CI]   ..... Day 40-49: GitHub Actions
    |-- Build with Maven
    |-- Run tests
    |-- Build Docker image  ..... Day 29-37: Docker
    |-- Push to DockerHub
    |-- Update K8s manifest
    |-- Commit back to Git
    |
[ArgoCD detects change] ........ Day 84-86: GitOps
    |
[ArgoCD syncs to EKS]  ........ Day 81-83: EKS
    |-- Rolling update
    |-- Health checks pass
    |-- HPA scales as needed ... Day 78-80: Helm (HPA, values)
    |
[Prometheus scrapes metrics] ... Day 73-77: Observability
    |-- Grafana dashboards
    |-- Alerts if something breaks
    |
[App is live with zero downtime]
```

Every block in this challenge connects to the next. This is what a DevOps pipeline looks like in production.

Task 6: Complete Teardown

Delete everything. This is the end of the EKS and ArgoCD block.

Delete ArgoCD applications:

- **argocd app delete bankapp --cascade -y**
- **argocd app delete monitoring --cascade -y 2>/dev/null**
- **argocd app delete envoy-gateway --cascade -y 2>/dev/null**
- **argocd app delete root-app --cascade -y 2>/dev/null**

The `--cascade` flag tells ArgoCD to delete all Kubernetes resources managed by each application.

Wait for cleanup:

- **kubectl get all -n bankapp 2>/dev/null**
- **kubectl get all -n monitoring 2>/dev/null**

```
karina@control-plane:~/devops/AI-BankApp-DevOps$ argocd app delete bankapp --cascade -y
application 'bankapp' deleted
karina@control-plane:~/devops/AI-BankApp-DevOps$ argocd app delete monitoring --cascade -y 2>/dev/null
argocd app delete envoy-gateway --cascade -y 2>/dev/null
argocd app delete root-app --cascade -y 2>/dev/null
application 'monitoring' deleted
application 'envoy-gateway' deleted
application 'root-app' deleted
karina@control-plane:~/devops/AI-BankApp-DevOps$ kubectl get all -n bankapp 2>/dev/null
kubectl get all -n monitoring 2>/dev/null
```

Destroy the EKS cluster with Terraform:

- **cd AI-BankApp-DevOps/terraform**
- **terraform destroy**

Confirm deletion (type `yes`). This takes 10-15 minutes.

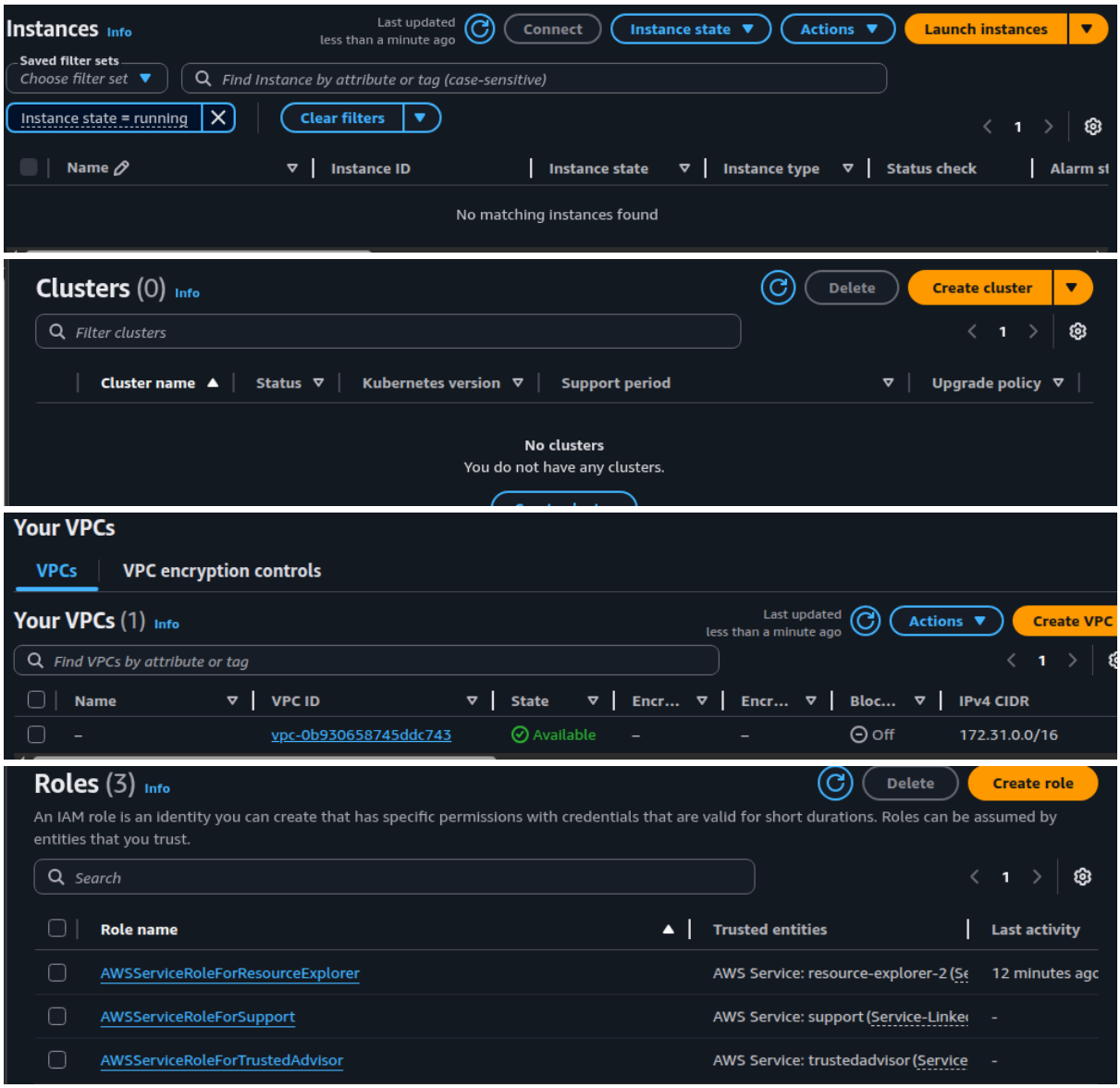
Verify in the AWS Console:

- EKS: no clusters

- EC2: no instances, no load balancers, no EBS volumes

- VPC: the `bankapp-eks` VPC is gone

- IAM: clean up roles with `eksctl` or `bankapp-eks` in the name

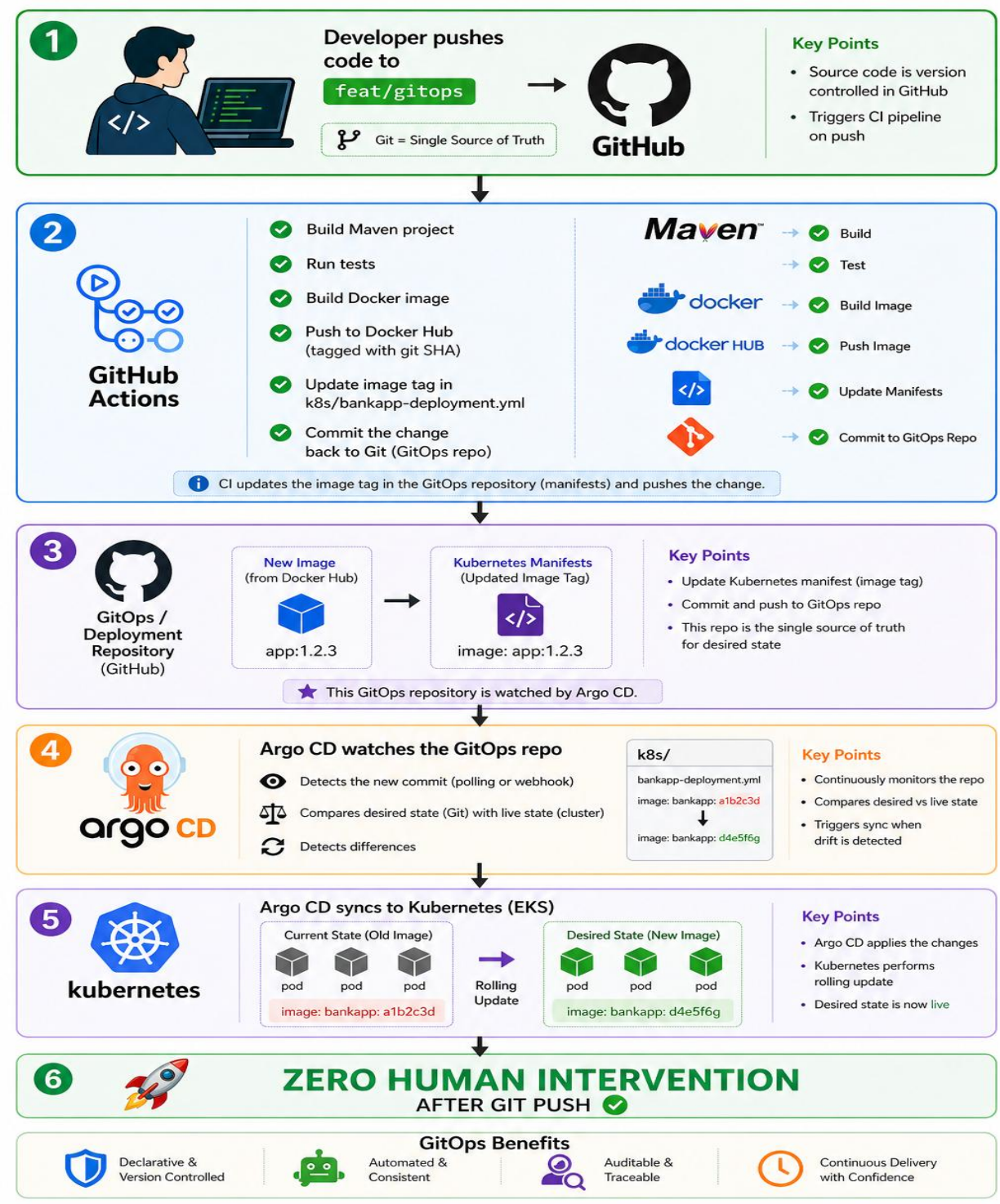


Final cost check: Review AWS Billing Dashboard. All EKS charges should stop within the hour.

Map the 3-day ArgoCD journey:

Day	What You Built
84	ArgoCD setup, first GitOps deploy, self-healing
85	Sync waves, rollbacks, App of Apps, notifications, RBAC
86	Full CI/CD pipeline, code-to-production, drift detection, teardown

The complete GitOps pipeline diagram (code -> CI -> Git -> ArgoCD -> EKS)



- GitHub Actions workflow explained step by step

yml

label

name: GitOps CI - Build & Push to DockerHub

trigger conditions: runs automatically when code is pushed to branch feat/gitops and changes are in src/, pom.xml, dockerfile also manual trigger

```
on:
push:
  branches: [feat/gitops]
  paths:
   - 'src/'
   - 'pom.xml'
   - 'Dockerfile'
workflow_dispatch:
```

give write permissions

```
    - permissions:
    - contents: write
```

environemnt variable used later

```
env:
DOCKERHUB_REPO: afroz25/ai-bankapp-eks
```

Creates a job named build-and-push runs on a fresh Ubuntu virtual machine

```
jobs:
build-and-push:
  name: Build, Test & Push Docker Image
  runs-on: ubuntu-latest

  steps:
   checkout code
   - name: Checkout code
     uses: actions/checkout@v4
```

setup jdk

```
- name: Set up JDK 21
  uses: actions/setup-java@v4
  with:
    java-version: '21'
    distribution: 'temurin'
    cache: 'maven'
```

build application:Cleans previous builds,Compiles code,Packages into JAR,Skips tests

```
- name: Build with Maven
  run: ./mvnw clean package -DskipTests -B
```

Executes unit tests, even if tests fail → workflow continues

```
- name: Run tests
  run: ./mvnw test -B
  continue-on-error: true
```

generate short tag of current commit, image is tagged with this id later

```
- name: Set image tag
  id: tag
  run: echo "sha_short=$(git rev-parse --short HEAD)" >> "$GITHUB_OUTPUT"
```

login to docker hub using github secrets

```
- name: Login to DockerHub
  uses: docker/login-action@v3
  with:
    username: ${{ secrets.DOCKERHUB_USERNAME }}
```

```
    password: ${{ secrets.DOCKERHUB_TOKEN }}

  Enables advanced Docker build features required for caching and multi-platform builds
  - name: Set up Docker Buildx
    uses: docker/setup-buildx-action@v3

  Builds Docker image from Dockerfile, Tags it: latest, Pushes to DockerHub
  Uses GitHub cache for faster builds
  - name: Build and Push Docker image
    uses: docker/build-push-action@v6
    with:
      context: .
      push: true
      tags: |
        ${{ env.DOCKERHUB_REPO }}:latest
        ${{ env.DOCKERHUB_REPO }}:${{ steps.tag.outputs.sha_short }}
      cache-from: type=gha
      cache-to: type=gha,mode=max

  Updates deployment YAML, replaces old image tag with new SHA tag
  - name: Update Kubernetes deployment manifest
    run: |
      sed -i "s|image: ${{ env.DOCKERHUB_REPO }}:.|image: ${{ env.DOCKERHUB_REPO
}}:${{ steps.tag.outputs.sha_short }}|" k8s/bankapp-deployment.yml

  Configures Git identity, adds modified file
  Commits the change with `[skip ci]` to avoid re-triggering
  - name: Commit updated manifest
```

```
      run: |
        git config user.name "github-actions[bot]"
        git config user.email "github-actions[bot]@users.noreply.github.com"
        git add k8s/bankapp-deployment.yml
        git diff --staged --quiet || git commit -m "ci: update bankapp image to ${{
steps.tag.outputs.sha_short }} [skip ci]"
        git push
```

- Key takeaways from the 3-day GitOps block

  - GitHub is single source of truth

  - Multiple application deployed using app of apps

  - Sync wave annotation orders to be given carefully

  - Complete pplication deployed using single `kubectl apply` command

  - ArgoCD keeps all applicaitons in check

  - `selfHeal` automatically syncs if any drift detected